

ESCUELA SUPERIOR POLITÉCNICA DE CHIMBORAZO

FACULTAD DE INFORMÁTICA Y ELECTRÓNICA

CARRERA DE TECNOLOGÍAS DE LA INFORMACIÓN

SISTEMA WEB DE MONITOREO Y GESTIÓN DE LA INFRAESTRUCTURA DE REDES - ESPOCH

DOCUMENTO DE ESTÁNDAR DE PROGRAMACIÓN Y GUÍA DE ESTILO

Convenciones de Nomenclatura, Arquitectura Limpia, Manejo Asíncrono y Seguridad de Código

Integrantes del Equipo:

- William Chisag
- Licet Ibarra
- Karen Maje
- Alejandro Rodríguez

Asignaturas y Docentes Responsables:

- Infraestructura TI: Ing. Tatiana Landivar
- Tecnología Web: Ing. Pedro Aguilar
- Conmutación y Enrutamiento: Ing. Juan Pazmiño
- Big Data: Ing. Geovanny Silva
- Diseño y Multimedia: Ing. Ramiro Bonilla

Período Académico: Quinto Semestre (PAO 5)

Documento Oficial de Ingeniería de Software (Versión 1.0)

Orellana - Ecuador

Octubre de 2026

Índice general

1. Metáfora de la Programación: La Fábrica de Precisión Mecánica **3**

1.1. Concepto de la Metáfora: La Fábrica de Precisión 3

2. Convenciones Estrictas de Nomenclatura **5**

2.1. Reglas Semánticas para Variables y Funciones 5

3. Estructura de Directorios y Modularidad (RNF-12) **7**

3.1. Estructura del Backend (`server/`) 7

3.2. Estructura del Frontend (`client/`) 8

4. Manejo Asíncrono y Tratamiento Centralizado de Excepciones **10**

4.1. Regla de Asincronía: Exclusividad de `async/await` 10

4.2. Manejo Centralizado de Excepciones en Express 11

5. Seguridad del Código y Validación Defensiva **12**

5.1. Prevención Inflexible de Inyección de Comandos en Linux 12

5.2. Validación de Datos en Entrada con Esquemas Zod 13

6. Estándar de Documentación de Código (JSDoc) **14**

7. Conclusiones **15**

Índice de tablas

2.1. Matriz de convenciones de nomenclatura en el proyecto 5

Capítulo 1

Metáfora de la Programación: La Fábrica de Precisión Mecánica

1.1 Concepto de la Metáfora: La Fábrica de Precisión

En la ingeniería de software moderna, el código fuente no es simplemente un conjunto de instrucciones ejecutables por un procesador, sino un documento formal que debe ser leído, comprendido y mantenido por personas. Conforme al requerimiento no funcional **RNF-12 (Mantenibilidad y modularidad)**, se adopta la metáfora de:

"La Fábrica de Mecanizado de Precisión e Instrumental Quirúrgico"

Bajo esta analogía de calidad:

- **Piezas Estandarizadas con Grabado Láser (Nomenclatura):** Cada tornillo, engranaje o sensor en la fábrica posee un código de serie y dimensiones exactas grabadas con láser (*camelCase*, *PascalCase*, etc.). Un mecánico no puede tomar una pieza genérica etiquetada como `x` o `data1` e instalarla en la línea de combustible; cada componente debe revelar con precisión milimétrica su función y tolerancia.
- **Células de Fabricación Aisladas (Arquitectura en Capas):** La sección de forja pesada (Daemon y red de bajo nivel) no está mezclada con la sala de pulido estético (Frontend React) ni con el almacén contable (Base de datos MongoDB). Cada estación cumple una única responsabilidad estricta y se comunica con las demás mediante pasarelas estandarizadas (APIs REST).
- **Válvulas de Seguridad y Fusibles (Manejo de Excepciones):** Si una sobrecarga de presión ocurre en una tubería secundaria (un timeout en una comprobación ICMP a una PC apagada), el fusible local se dispara de forma controlada sin reventar la caldera principal de la fábrica (el daemon de monitoreo continúa vigilando al resto de los 30 dispositivos sin detenerse jamás, cumpliendo RNF-03 y RNF-15).
- **Control de Calidad en Banda (Linters y Validaciones):** Antes de embalar cualquier instrumento, pasa por un sensor óptico (ESLint y esquemas Zod) que detecta cualquier fisura o desviación milimétrica en el grosor del material antes de que llegue a manos del cliente.

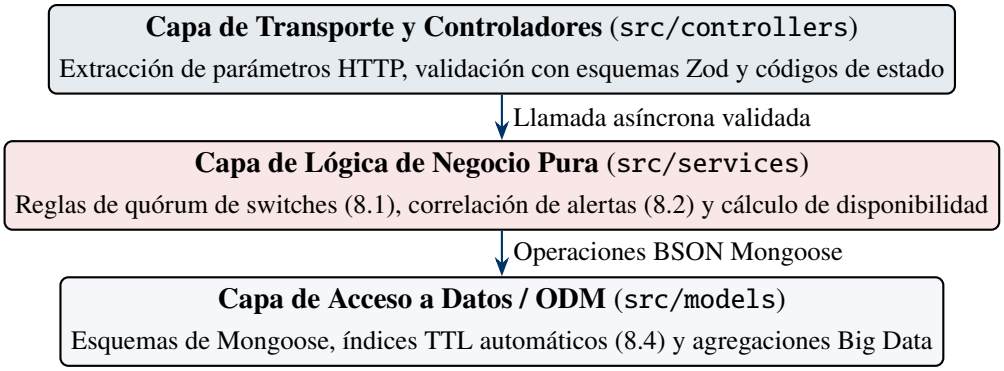


Figura 1.1: Flujo unidireccional estricto entre capas en la arquitectura limpia del backend

Capítulo 2

Convenciones Estrictas de Nomenclatura

Para evitar ambigüedades y garantizar la homogeneidad del código desarrollado por los cuatro integrantes del equipo, se establecen las siguientes convenciones de tipografía de identificadores:

Tabla 2.1: Matriz de convenciones de nomenclatura en el proyecto

Estilo Tipográfico	Elementos del Código	Ejemplos Correctos	Ejemplos Incorrectos (Prohibidos)

camelCase	Variables locales, propiedades de objetos JSON, nombres de funciones y métodos	`ipAddress`
		`activeDeviceCount`
		`calculateAvailability()`
		`isServiceRunning`
			`ip_address` (snake)
			`ActiveDeviceCount`
			`CalcAvail()`
			`flag1`

PascalCase	Componentes React, clases, modelos de Mongoose, interfaces y tipos Zod	`NetworkTopologyGraph`
		`DeviceCard`
		`DeviceModel`
		`UserCreateSchema`
			`networkTopologyGraph`
			`device_card`
			`deviceModel`

UPPER_SNAKE_CASE	Constantes globales inmutables, variables de entorno y límites de seguridad	`DEFAULT_PING_TIMEOUT_MS`
		`MAX_CONCURRENT_TASKS`
		`JWT_ACCESS_SECRET`
		`MIN_PING_INTERVAL_SEC`
			`defaultPingTimeout`
			`maxTasks`
			`jwtSecret`

kebab-case	Nombres de archivos JS/JSX, directorios, rutas de endpoints REST y ramas Git	`device-controller.js`
		`network-scanner.service.js`
		`/api/v1/network-devices`
		`feature/auth-jwt-refresh`
			`DeviceController.js`
			`networkScanner.js`
			`/api/v1/networkDevices`
			`Feature/Auth`

2.1 Reglas Semánticas para Variables y Funciones

- **Variables Booleanas:** Deben iniciar obligatoriamente con prefijos verbales auxiliares que denoten estado afirmativo o negativo:

```
// Correcto:
const isReachable = true;
const hasCustomIp = false;
const canEditTopology = userRole === 'admin';

// Prohibido:
const status = true; // Que tipo de estado?
```

```
const edit = true;   // Es accion o bandera?
const reachable = true;
```

- **Funciones y Métodos:** Deben iniciar con un verbo infinitivo en inglés que describa de manera inequívoca la acción ejecutada:
  - `get...` / `fetch...`: Obtención de datos sin modificar el estado (ej. `getDeviceById`).
  - `create...` / `register...`: Creación o registro de nuevas entidades (ej. `registerDevice`).
  - `update...`: Modificación de entidades existentes (ej. `updateDeviceConfiguration`).
  - `delete...` / `remove...`: Eliminación o baja lógica (ej. `deactivateUserAccount`).
  - `check...` / `verify...`: Comprobación de condición booleana o prueba de red (ej. `checkHostAvailability`).
- **Prohibición Absoluta de Nombres Mágicos o de una Letra:** Quedan terminantemente prohibidos identificadores como `x`, `d`, `arr`, `temp`, `obj`, salvo el índice `i` en bucles numéricos simples de conteo.

Capítulo 3

Estructura de Directorios y Modularidad (RNF-12)

El proyecto debe organizarse en dos raíces claramente desacopladas: el directorio `server/` (Backend y Daemon) y el directorio `client/` (Frontend React).

3.1 Estructura del Backend (`server/`)

```
server/
|-- src/
|   |-- config/
|   |   |-- database.js            # Conexion a MongoDB con
   reconexion automatica
|   |   |-- environment.js         # Carga y validacion de
   variables de entorno
|   |   `-- constants.js           # Limites seguros (guardrails)
   de la Seccion 8.7
|   |-- controllers/
|   |   |-- auth.controller.js     # Login, logout, refresh token,
   recuperacion
|   |   |-- device.controller.js   # CRUD de inventario, bajas
   logicas
|   |   |-- alert.controller.js    # Listado, reconocimiento y
   cierre de alertas
|   |   |-- metric.controller.js   # Estadisticas de disponibilidad
    (RF-11)
|   |   `-- config.controller.js   # Configuracion de intervalos y
   restablecimiento
|   |-- daemon/
|   |   |-- ping-worker.js         # Daemon autonomo ICMP con p-
   limit a 15 tareas
|   |   |-- arp-scanner.js         # Descubrimiento nativo con arp-
   scan en Linux
|   |   `-- quorum-evaluator.js    # Evaluacion de quorums del 70%
   en switches (8.1)
|   |-- middlewares/
|   |   |-- auth.middleware.js     # Verificacion de firma de
   Access Token JWT
```

```
|   |   |-- role.middleware.js      # Verificacion de roles RBAC (
    admin vs operador)
|   |   |-- validate.middleware.js # Middleware generico de
    validacion Zod
|   |   `-- error.middleware.js    # Capturador centralizado de
    excepciones
|   |-- models/
|   |   |-- device.model.js        # Esquema polimorfico de
    dispositivos
|   |   |-- check.model.js         # Comprobaciones brutas con
    indice TTL 30d
|   |   |-- event.model.js         # Eventos historicos con indice
    TTL 90d
|   |   |-- alert.model.js         # Alertas operativas (abierta,
    reconocida, cerrada)
|   |   |-- user.model.js          # Usuarios, roles y password
    hashes
|   |   `-- audit.model.js         # Auditoria inmutable de
    acciones administrativas
|   |-- services/
|   |   |-- auth.service.js        # Logica de generacion y
    rotacion de tokens
|   |   |-- device.service.js      # Prevencion de duplicados y
    cambios de IP
|   |   `-- topology.service.js    # Calculo de relaciones
    confirmadas e inferidas
|   `-- app.js                     # Inicializacion de Express y
    rutas base
|-- tests/                         # Pruebas unitarias y de
    integracion con Jest
|-- .env.example
|-- package.json
`-- server.js                      # Punto de entrada HTTP y
    arranque del daemon
```

3.2 Estructura del Frontend (`client/`)

```
client/
|-- src/
|   |-- assets/                    # Logotipos oficiales ESPOCH y
    vectores SVG
|   |-- components/
|   |   |-- common/                # Botones accesibles, modales,
    badges de estado
```

```
|   |   |-- layout/                 # Barra superior con campana,
    menu lateral
|   |   |-- dashboard/              # Tarjetas de resumen NOC y
    actividad reciente
|   |   |-- topology/               # Lienzo interactivo React Flow
    y nodos de red
|   |   `-- inventory/              # Tablas con filtros combinados
    y paginacion
|   |-- context/
|   |   |-- AuthContext.jsx         # Estado de sesion, rol y auto-
    refresh de token
|   |   `-- AlertContext.jsx        # Notificaciones flotantes (
    toasts) y sonido
|   |-- hooks/
|   |   |-- useDevices.js           # Polling controlado a /api/
    devices cada 5s
|   |   `-- useInterval.js          # Temporizador declarativo
    seguro contra memory-leaks
|   |-- pages/
|   |   |-- LoginPage.jsx
|   |   |-- DashboardPage.jsx
|   |   |-- TopologyPage.jsx
|   |   |-- InventoryPage.jsx
|   |   `-- SettingsPage.jsx
|   |-- services/
|   |   `-- api.js                  # Instancia Axios con
    interceptor de JWT
|   |-- App.jsx
|   `-- main.jsx
|-- tailwind.config.js
|-- vite.config.js
`-- package.json
```

Capítulo 4

Manejo Asíncrono y Tratamiento Centralizado de Excepciones

El sistema ejecuta continuas operaciones de red y base de datos. Para evitar bloqueos del bucle de eventos (*Event Loop*) y garantizar respuestas rápidas (RNF-01), se establecen las siguientes reglas:

4.1 Regla de Asincronía: Exclusividad de `async/await`

- Queda terminantemente prohibido el anidamiento descontrolado de llamadas `.then().then().catch(` (antipatrón conocido como *Callback Hell*).
- Todas las operaciones asíncronas deben declararse con `async` y consumirse mediante `await`.
- Para operaciones concurrentes independientes (ejemplo: comprobar varios dispositivos a la vez), se debe utilizar **`Promise.allSettled()`** en combinación con un limitador de concurrencia (`p-limit`), impidiendo que la falla de un dispositivo cancele las comprobaciones de los demás (RF-04.09 y RF-05.10).

```
// Ejemplo correcto de ejecucion concurrente controlada:
import pLimit from 'p-limit';

export async function pingDeviceBatch(devices, timeoutMs) {
  // Limite estricto de 15 comprobaciones simultaneas segun RNF-15
  const limit = pLimit(15);

  const checkPromises = devices.map(device =>
    limit(() => pingSingleDevice(device.ipAddress, timeoutMs))
  );

  // allSettled garantiza que ningun error individual aborte a los
     otros
  const results = await Promise.allSettled(checkPromises);
  return results.map(res => res.status === 'fulfilled' ? res.value
     : null);
}
```

4.2 Manejo Centralizado de Excepciones en Express

Los controladores de la API no deben capturar errores para simplemente imprimir `console.log(error)`. Deben utilizar una función envoltorio asíncrona (*asyncHandler*) que canalice automáticamente cualquier fallo hacia el middleware central de errores:

```
// src/middlewares/async-handler.js
export const asyncHandler = (fn) => (req, res, next) => {
  Promise.resolve(fn(req, res, next)).catch(next);
};

// src/controllers/device.controller.js
export const getDeviceDetails = asyncHandler(async (req, res) => {
  const { id } = req.params;
  const device = await deviceService.findDeviceById(id);

  if (!device) {
    return res.status(404).json({
      success: false,
      message: `Dispositivo con identificador ${id} no encontrado.`
    });
  }

  return res.status(200).json({ success: true, data: device });
});
```

Capítulo 5

Seguridad del Código y Validación Defensiva

Conforme a los estándares de desarrollo seguro ISO/IEC 27001, se aplican técnicas de codificación defensiva en todas las capas:

5.1 Prevención Inflexible de Inyección de Comandos en Linux

Dado que el sistema ejecuta utilitarios del sistema operativo (`ping`, `arp-scan`, `nmap`), **queda terminantemente prohibido utilizar `child_process.exec()` con cadenas concatenadas**, ya que un atacante podría enviar una dirección IP manipulada como `"192.168.1.1; rm -rf /"`.

Estándar de Seguridad Obligatorio:

1. Validar rigurosamente la dirección IP con una expresión regular RFC antes de cualquier invocación.
2. Utilizar de manera exclusiva **`child_process.spawn()`** o **`execFile()`**, pasando los argumentos como un vector de cadenas independientes (*array*), neutralizando cualquier intento de inyección de comandos en la shell:

```
import { spawn } from 'child_process';
import net from 'net';

export function executeSafePing(ipAddress, timeoutSeconds = 3) {
  return new Promise((resolve, reject) => {
    // 1. Validacion defensiva estricta de IP antes de tocar el
       sistema
    if (!net.isIP(ipAddress)) {
      return reject(new Error(`Direccion IP no valida: ${ipAddress
        }`));
    }

    // 2. Invocacion segura pasando parametros en array sin invocar
        shell
    const pingProcess = spawn('ping', ['-c', '1', '-W', String(
       timeoutSeconds), ipAddress]);
```

```
    pingProcess.on('close', (exitCode) => {
      // Codigo 0 = respuesta exitosa; Codigo 1 = timeout / host
         inalcanzable
      resolve({ isAlive: exitCode === 0, exitCode });
    });

    pingProcess.on('error', (err) => {
      reject(new Error(`Fallo interno del binario ping: ${err.
         message}`));
    });
  });
}
```

5.2 Validación de Datos en Entrada con Esquemas Zod

Toda petición POST o PUT debe validarse contra un esquema inmutable antes de procesar la lógica de negocio:

```
import { z } from 'zod';

export const UpdateConfigSchema = z.object({
  pingIntervalSec: z.number().int().min(10).max(300),
  discoveryIntervalMin: z.number().int().min(2).max(60),
  pingTimeoutMs: z.number().int().min(1000).max(10000),
  consecutiveFailuresForDown: z.number().int().min(2).max(10),
  consecutiveSuccessesForUp: z.number().int().min(1).max(5),
  maxConcurrentPings: z.number().int().min(5).max(50)
});
```

Capítulo 6

Estándar de Documentación de Código (JSDoc)

Para garantizar la analizabilidad de la norma ISO/IEC 25010, todas las funciones exportadas, clases y modelos deben contener un bloque JSDoc estructurado:

```
/**
 * Evalua el estado de quorun colectivo para switches no
    administrables sin IP.
 * Si mas del 70% de los dispositivos dependientes estan caidos, se
     genera
 * una alerta consolidada de infraestructura (Seccion 8.1 de la
    especificacion).
 *
 * @param {string} switchId - Identificador interno unico del
    switch padre.
 * @param {Array<Object>} connectedDevices - Lista de dispositivos
    conectados al switch.
 * @returns {Promise<Object>} Resultado de la evaluacion y estado
    de la alerta.
 * @throws {DatabaseError} Si falla la consulta a la coleccion de
    alertas activas.
 */
export async function evaluateSwitchQuorum(switchId,
   connectedDevices) {
  // Implementacion...
}
```

Capítulo 7

Conclusiones

El cumplimiento sistemático de este estándar de programación garantiza que la solución de software no sufra de degradación técnica con el paso del tiempo. Al aplicar nomenclatura homogénea, modularidad estricta, manejo asíncrono con límites de concurrencia y validaciones de seguridad infranqueables, el equipo de desarrollo de la ESPOCH entregará una plataforma de monitoreo robusta, escalable y mantenible.